COP509
Natural Language Processing

Abstractive and Extractive Summarisation

Prof. Georgina Cosma

Department of Computer Science, Loughborough University

Outline

1. What is text summarisation?
2. Extractive summarisation
3. Abstractive summarisation
4. Architectures behind modern systems
5. Evaluation metrics (with formulas and worked examples)
6. Key takeaways

These slides are designed to be read independently. Each concept is fully explained so you can follow along even if you missed the lecture.

Part 1

What is text summarisation?

Text summarisation

Text summarisation is the task of producing a **shorter version** of a document (or collection of documents) that preserves the most important information.

The goal is to reduce the amount of text a reader must process while keeping the core meaning and key facts intact.

Why summarisation matters

Billions of documents are produced daily. Humans cannot read them all. Automatic summarisation appears in news digests, legal briefs, clinical record review, search-engine snippets, meeting minutes, and any application where time-to-information matters.

An example of summarisation

Original (80 words)

A new clinical study involving 2,000 participants has found that regular 30-minute walks five times a week reduce the risk of type 2 diabetes by up to 40% over a five-year period. The study, published in *The Lancet*, also reported improvements in blood pressure and sleep quality among participants. Researchers noted that the benefits were observed across all age groups but were most pronounced in participants over 50.

Summary (25 words)

Five 30-minute walks per week cut type 2 diabetes risk by up to 40% and improve blood pressure and sleep, especially in over-50s, a Lancet study finds.

The summary preserves the main finding, magnitude, and key caveat, whilst dropping details (sample size, study duration) that are less essential.

Two fundamentally different approaches

There are two main ways to build a summarisation system. They differ in how the output is produced.

	Extractive	**Abstractive**
How it works	Selects existing sentences	Generates new text
Analogy	Using a highlighter	Writing notes in your own words
Output words	Only words from the source	May include new words
Risk profile	Faithful by construction	Can hallucinate

The rest of the lecture covers how each approach works, and how we evaluate whether a summary is any good.

Part 2

Extractive summarisation

Extractive summarisation — the core idea

An extractive summariser picks the most important sentences directly from the source document and concatenates them. It does not generate any new words.

The central question

Given a document with n sentences, which subset of k sentences (where $k < n$) best represents the document?

All extractive methods boil down to one problem: **how do we score sentences by importance?** Different methods propose different scoring schemes. We will see three.

Method 1: TF-IDF scoring

The simplest extractive method scores each sentence by the importance of the words it contains, using **TF-IDF** (Term Frequency – Inverse Document Frequency).

- **Term Frequency (TF)**: how often a word appears in the document.
- **Inverse Document Frequency (IDF)**: how rare a word is across many documents. Common words ("the", "is") have low IDF; specialist words have high IDF.
- **TF-IDF = TF × IDF**. High for words frequent here but rare elsewhere → informative.

Give each sentence a score equal to the sum of its words' TF-IDF values. Keep the top-k scoring sentences.

Intuition

A sentence using many rare, topic-specific words is probably important. A sentence full of common words probably isn't.

Method 2: TextRank (graph-based)

TextRank treats the document as a **graph** and uses PageRank (the original Google algorithm) to find important sentences.

Building the graph:

- Each sentence becomes a *node*.
- Two sentences are connected by an *edge* if they are similar (shared vocabulary).
- The edge weight is the similarity score (often cosine similarity).

Running PageRank: a sentence is considered important if it is similar to many other important sentences. PageRank computes this by iteratively passing "votes" along edges until scores stabilise.

Intuition

The most "central" sentence — the one most connected to the rest of the document — is probably the best summary of it.

Method 3: BERTSum (neural)

TF-IDF and TextRank treat words as symbols and miss meaning. Modern extractive summarisers use pre-trained language models like BERT.

BERTSum:

- Insert a special `[CLS]` token before every sentence.
- Pass the document through BERT to get a contextual vector for each `[CLS]` token.
- Train a classifier on each `[CLS]` vector to predict **include / exclude**.
- At test time, keep sentences with high "include" probability.

Why this is better

BERT understands that "catastrophic flooding" and "severe inundation" refer to the same idea, so BERTSum selects sentences by meaning, not by surface word overlap.

Extractive summarisation — worked example

Source document

(1) The Amazon rainforest covers over 5.5 million km^2. (2) It is home to roughly 10% of all species on Earth. (3) Deforestation rates have increased sharply since 2019. (4) Scientists warn that continued loss could trigger a tipping point. (5) Brazil accounts for about 60% of the rainforest.

A TF-IDF scorer computes an importance score for each sentence and selects the top 3: sentences **1, 3, 4**.

Resulting extractive summary

The Amazon rainforest covers over 5.5 million km^2. Deforestation rates have increased sharply since 2019. Scientists warn that continued loss could trigger a tipping point.

Extractive — strengths and weaknesses

Strengths	Weaknesses
Grammatically correct	Reads as disjointed
Factually faithful — nothing invented	Redundancy possible
Fast and easy to implement	Cannot paraphrase or generalise
Easy to explain and audit	Whole sentences only

Extractive summarisation is the safer choice when **faithfulness matters more than fluency** — e.g. legal or clinical text where inventing information is unacceptable.

Part 3

Abstractive summarisation

Abstractive summarisation — the core idea

An abstractive summariser **generates** the summary as new text, rephrasing and combining information from the source. The output may contain words that never appeared in the original.

This is much more flexible than extraction: the model can merge two sentences, paraphrase difficult terms, or condense a paragraph into a single clause.

The trade-off

Abstractive summaries are more fluent and natural, but they introduce a new risk: the model may generate content not supported by the source. This is called **hallucination**, and it is the central challenge of abstractive summarisation.

How abstractive models are built

Modern abstractive summarisers are **pre-trained Transformer** models. They come in two families:

Encoder–decoder models — designed for sequence-to-sequence tasks:

- **BART**: pre-trained by corrupting text and reconstructing it.
- **T5**: every NLP task framed as "text in $\rightarrow$ text out".
- **PEGASUS**: pre-trained by masking whole sentences.

Decoder-only models — the Large Language Models (LLMs):

- **GPT-4, Claude, LLaMA**: can summarise via a prompt like "Summarise this: . . . "
- No dedicated summarisation training required at scale.

Abstractive summarisation — worked example

Source document (same as before)

(1) The Amazon rainforest covers over 5.5 million km^2. (2) It is home to roughly 10% of all species on Earth. (3) Deforestation rates have increased sharply since 2019. (4) Scientists warn that continued loss could trigger a tipping point. (5) Brazil accounts for about 60% of the rainforest.

Abstractive summary (e.g. PEGASUS or GPT-4)

The Amazon, spanning 5.5 million km^2 and hosting 10% of the world's species, faces accelerating deforestation that scientists fear could reach a tipping point.

The model merged sentences 1+2, paraphrased sentence 3 as "accelerating", and synthesised sentence 4. More fluent and information-dense than the extractive version — at the cost of introducing words not in the source.

Hallucination — the main risk of abstraction

A **hallucination** is information in the summary that is not supported by the source.

Hallucination example

Source: "The company reported revenue of $4.2 billion."

Summary: "The company reported **record** revenue of **$4.5 billion**."

Two problems:

- "record" is **unsupported** — the source never said the figure was a record.
- "$4.5 billion" **contradicts** the source ($4.2 billion).

A hallucinated summary may read perfectly and score well on simple metrics, yet be factually wrong. This is why we need **faithfulness metrics** — see Part 5.

Abstractive — strengths and weaknesses

Strengths	Weaknesses
Fluent, natural-sounding output	Can hallucinate facts
Can merge, compress, and generalise	Computationally expensive
Close to human-written summaries	Harder to evaluate
Works across languages and styles	Can subtly change meaning

Abstractive summarisation is the right choice when **fluency and compression matter**, and the domain allows occasional imperfect paraphrasing (e.g. news, general text).

Part 4

Architectures behind modern systems

What does an encoder do?

An **encoder** is a neural network that **reads and understands** a piece of input text. Its job is to produce a rich numerical representation of the input — it does *not* produce any output text.

How it works:

- The input (e.g. a document) is tokenised into words/subwords.
- Each token is converted to a vector (an embedding).
- The encoder applies **bidirectional self-attention**: every token can attend to every other token — both to the left and to the right.
- Output: one *contextual vector* per input token, capturing its meaning in context.

The encoder's purpose

Turn text into vectors that capture meaning. Useful whenever we need to **understand** something about the input: classify it, extract information, or pass it on to something else.

What does a decoder do?

A **decoder** is a neural network that **generates text**, one token at a time. Its job is to produce output.

How it works:

- At each step, it predicts the next token given all the tokens generated so far.
- It uses **causal self-attention**: each token can only attend to *previous* tokens, never future ones (you cannot see what you have not yet written).
- This repeats until an end-of-sequence token is produced.

Two flavours of decoder:

- **With cross-attention to an encoder** (used in encoder–decoder models): the decoder attends to the encoder's output vectors when generating each token.
- **Without an encoder** (decoder-only models): the decoder attends only to its own previous tokens.

Three architecture families

Different tasks need different architectures. There are three main families.

Family	Components	Designed for	Example models
Encoder-only	Encoder only	Understanding: classification, extraction, retrieval	BERT, RoBERTa, DeBERTa
Decoder-only	Decoder only	Generation: continuation, chat, open-ended writing	GPT-4, Claude, LLaMA, Mistral
Encoder–decoder	Encoder + Decoder	Transformation: input → different output	BART, T5, PEGASUS, mT5

Rule of thumb

"Input → label" → encoder-only. "Continue this text" → decoder-only.
"Input text → different output text" → traditionally encoder–decoder.

Summarisation: encoder–decoder OR decoder-only?

Summarisation is a "text → different text" task, so historically it used **encoder–decoder** models (BART, T5, PEGASUS). But today, **decoder-only** LLMs (GPT-4, Claude) also summarise well. Why?

The puzzle resolved

In a decoder-only model, the source document sits in the **context window** as part of the prompt. The model's self-attention layers read and process it before generating any summary tokens. So the "understanding" work that the encoder would do is done *implicitly* by the decoder's own self-attention.

In other words: the decoder-only model does not skip the encoding step. It just does it inside the same stack of layers, rather than in a separate encoder network.

Why encoder–decoder models still exist: they are more efficient for long inputs, easier to fine-tune on small datasets, and let you optimise the encoder and decoder separately.

Models used in summarisation

Model	Architecture	Notes
BERTSum	Encoder-only	Uses BERT as encoder; classifies each sentence as include/exclude. Used for **extractive** summarisation.
BART	Encoder–decoder	Pre-trained by corrupting text and reconstructing it. Strong abstractive summariser.
T5	Encoder–decoder	"Text in $\rightarrow$ text out" for every task. Prompt: `summarize: ...`
PEGASUS	Encoder–decoder	Pre-training masks whole sentences, forcing the model to reconstruct them — closely mimics summarisation.
GPT-4	Decoder-only	General-purpose LLM. Summarises via a prompt; zero-shot or few-shot.
Claude	Decoder-only	Large context window (useful for long documents).
LLaMA, Mistral	Decoder-only	Open-weights LLMs that can be fine-tuned for summarisation.

Which architecture to pick when

Prefer encoder–decoder	Prefer decoder-only (LLM)
Need dedicated summarisation training (e.g. PEGASUS's masked-sentence pre-training)	Zero-shot or few-shot setting; no fine-tuning budget
Fine-tuning on a small domain-specific dataset	Combining summarisation with other tasks in one prompt
Efficiency matters; long documents in production	Compute available; best quality wanted

In practice, many pipelines are **hybrid**: an encoder-only or encoder–decoder model extracts candidate salient content, and an LLM polishes it into a final summary.

Part 5

Evaluation metrics

Why is evaluating summaries hard?

A single document can be summarised correctly in thousands of different ways. Two human-written summaries of the same article may share almost no words while conveying the same meaning.

Three valid summaries of the same article

"The dog was happy."
"The canine was joyful."
"A pleased hound."

A good evaluation metric must handle this variation, and ideally capture three separate qualities:

- **Content coverage** — does the summary contain the key information?
- **Fluency** — is the text grammatical and readable?
- **Faithfulness** — is every claim supported by the source?

No single metric captures all three, which is why we report several.

ROUGE — overview

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is the most widely used automatic summarisation metric.

> **Core idea**
>
> Compare the generated summary against a human-written reference summary by counting how many n-grams (word sequences of length n) they share.

Three main variants:

- **ROUGE-1**: unigram overlap (individual words).
- **ROUGE-2**: bigram overlap (two-word phrases).
- **ROUGE-L**: longest common subsequence (allowing gaps).

ROUGE — formulas

For a candidate summary C and a reference summary R, counting n-grams of length n:

ROUGE-N precision, recall, F_1

$$\text{Recall} = \frac{\#\text{matching } n\text{-grams}}{\#\ n\text{-grams in } R}$$

$$\text{Precision} = \frac{\#\text{matching } n\text{-grams}}{\#\ n\text{-grams in } C}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Plain English: *Recall* asks "how much of the reference did we capture?". *Precision* asks "how much of what we wrote was relevant?". F_1 is the harmonic mean — rewards being good at both.

ROUGE-1 — worked example

Reference and candidate

Reference: *The cat sat on the mat.*
Candidate: *The cat was sitting on a mat.*

Step 1. Tokenise into unigrams.
Reference: {the, cat, sat, on, the, mat} — 6 tokens.
Candidate: {the, cat, was, sitting, on, a, mat} — 7 tokens.

Step 2. Count matches. Words in both: {the, cat, on, mat} → **4 matches**.

Step 3. Apply the formulas.

$$\text{Recall} = \frac{4}{6} \approx 0.67 \qquad \text{Precision} = \frac{4}{7} \approx 0.57 \qquad F_1 = 2 \cdot \frac{0.67 \cdot 0.57}{0.67 + 0.57} \approx \mathbf{0.62}$$

ROUGE-2 — worked example

Using the same reference and candidate, now count **bigrams** (consecutive word pairs).

Step 1. Extract bigrams.
Reference: {the cat, cat sat, sat on, on the, the mat} — 5 bigrams.
Candidate: {the cat, cat was, was sitting, sitting on, on a, a mat} — 6 bigrams.

Step 2. Count matches. Only {the cat} is shared → **1 match**.

Step 3. Apply the formulas.

$$\text{Recall} = \frac{1}{5} = 0.20 \qquad \text{Precision} = \frac{1}{6} \approx 0.17 \qquad F_1 \approx \mathbf{0.18}$$

ROUGE-2 is much lower than ROUGE-1 because the phrasing differs substantially, even though the meaning is similar. *n*-gram metrics are sensitive to exact wording.

ROUGE-L — worked example

ROUGE-L uses the **Longest Common Subsequence (LCS)**: the longest sequence of words appearing in both summaries *in the same order*, but not necessarily contiguous.

ROUGE-L

$$\text{Recall}_L = \frac{\text{LCS}(C, R)}{\#\text{words in } R} \qquad \text{Precision}_L = \frac{\text{LCS}(C, R)}{\#\text{words in } C}$$

Finding the LCS: the longest subsequence appearing in order in both is *the, cat, on, mat* (length 4). Gaps between words are allowed.

$$\text{Recall}_L = \tfrac{4}{6} \approx 0.67 \qquad \text{Precision}_L = \tfrac{4}{7} \approx 0.57 \qquad F_1 \approx \mathbf{0.62}$$

ROUGE — limitations

ROUGE is the de-facto standard, but has serious weaknesses.

- **Surface-level only.** "The canine was joyful" vs "The dog was happy" share no words $\rightarrow$ ROUGE $= 0$ despite identical meaning.
- **All matches equal.** Matching "and" counts the same as matching a key technical term.
- **Favours extractive systems.** Extracted sentences copy source phrases exactly, giving an automatic n-gram overlap advantage.
- **Cannot detect hallucination.** A fluent but factually wrong summary can still score highly.
- **Reference-dependent.** Quality depends entirely on the quality of the human reference.

Useful for comparability with prior work, but should never be the only metric reported.

BERTScore — overview

BERTScore fixes ROUGE's biggest weakness by using **contextual embeddings** from BERT instead of exact string matching.

Core idea

Two words are "similar" if their BERT embeddings are close in vector space. This captures synonymy (happy $\approx$ joyful) and context ("bank" in finance $\neq$ "bank" in geography).

Procedure:

1. Encode each token in reference and candidate using BERT.
2. Compute **cosine similarity** between every pair of reference and candidate tokens.
3. For each reference token, take its maximum similarity against any candidate token.
4. Aggregate into precision, recall, F_1.

BERTScore — formulas

Let $\mathbf{x}_i$ be the BERT embedding of token i in reference R, and $\hat{\mathbf{x}}_j$ the embedding of token j in candidate C. Cosine similarity is $\cos(\mathbf{x}_i, \hat{\mathbf{x}}_j)$.

BERTScore precision, recall, F_1

$$R_{\text{BERT}} = \frac{1}{|R|} \sum_{\mathbf{x}_i \in R} \max_{\hat{\mathbf{x}}_j \in C} \cos(\mathbf{x}_i, \hat{\mathbf{x}}_j)$$

$$P_{\text{BERT}} = \frac{1}{|C|} \sum_{\hat{\mathbf{x}}_j \in C} \max_{\mathbf{x}_i \in R} \cos(\mathbf{x}_i, \hat{\mathbf{x}}_j)$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

Plain English: for recall, each reference token finds its best match in the candidate; for precision, vice versa.

BERTScore — worked example

Two summaries expressing the same meaning

Reference: *The dog was happy.*
Candidate: *The canine was joyful.*

ROUGE-1: only "the" and "was" match $\rightarrow 2/4 = 0.50$ (modest despite identical meaning).

BERTScore cosine similarities (illustrative):

Reference token	Best candidate match	Cosine similarity
the	the	1.00
dog	canine	**0.91**
was	was	1.00
happy	joyful	**0.92**

$$R_{\text{BERT}} = \frac{1}{4}(1.00 + 0.91 + 1.00 + 0.92) \approx \mathbf{0.96}$$

Where ROUGE misses the semantic match, BERTScore catches it.

METEOR — a metric that knows about synonyms

METEOR sits between ROUGE and BERTScore: it allows matches beyond exact words but does not need a neural model.

Four matching levels, applied in order:

- **Exact match**: *cat* $\leftrightarrow$ *cat*
- **Stem match**: *running* $\leftrightarrow$ *runs* (both stem to *run*)
- **Synonym match**: *happy* $\leftrightarrow$ *joyful* (via WordNet)
- **Paraphrase match**: *car crash* $\leftrightarrow$ *vehicle collision*

METEOR

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Penalty}) \qquad F_{\text{mean}} = \frac{10 \cdot P \cdot R}{R + 9P}$$

F_{mean} is a recall-weighted F-score. The **fragmentation penalty** reduces the score if matched words are scattered rather than in contiguous chunks.

Faithfulness metrics — motivation

ROUGE, BERTScore, and METEOR all compare the summary against a **reference** summary. None check whether the summary is supported by the **source**.

This matters because a summary can:

- score well against a reference summary, and
- still contain hallucinated information relative to the source.

What faithfulness metrics ask

Ignoring the reference entirely — is every claim in the generated summary actually supported by the source document?

They answer this using **Natural Language Inference (NLI)**: a classifier trained to judge whether one sentence *entails*, *contradicts*, or is *neutral* with respect to another.

Faithfulness metrics — FactCC and QuestEval

FactCC

- A BERT-based NLI classifier.
- *Premise* = source document. *Hypothesis* = summary sentence.
- Output: *consistent* (supported) or *inconsistent* (not supported).
- Final score: percentage of summary sentences classified as consistent.

QuestEval

- Automatically generate questions from the summary.
- Answer each question using the source.
- Answer from source matches answer from summary $\rightarrow$ faithful.
- Mismatch $\rightarrow$ hallucination detected, localised to a specific claim.

Faithfulness — worked example

Source and summary

Source: "Drug X reduced LDL cholesterol by 22% with no serious adverse events over **12 months**."

Summary: "Drug X reduced LDL cholesterol by 22% and was declared safe over a **two-year period**."

ROUGE / BERTScore: both score this summary high — most words and meaning align.
FactCC: NLI classifier reads (source, summary) and returns *inconsistent* — "two-year period" is contradicted by "12 months".
QuestEval: Q "How long was Drug X studied?". Summary says *two years*; source says *12 months*. Mismatch → hallucination flagged.

Takeaway

Only faithfulness metrics catch this error. ROUGE and BERTScore pass it.

Metrics summary

Metric	Measures	Key limitation
ROUGE	Surface *n*-gram overlap with reference	Misses synonyms; favours extractive
BERTScore	Semantic similarity with reference	Computationally heavy; needs BERT
METEOR	Matches + synonyms + morphology	Needs WordNet; less used recently
FactCC	Summary consistency with source	Binary; misses nuance
QuestEval	Summary consistency via QA	Complex multi-step pipeline
Human eval	All dimensions	Expensive; low agreement

Best practice for reporting

Report **ROUGE** for comparability, **BERTScore** for semantic quality, and a **faithfulness metric** for abstractive systems.

Part 6

Key takeaways

Key takeaways

1. **Extractive** selects existing sentences (faithful but disjointed). **Abstractive** generates new text (fluent but risks hallucination).
2. **TF-IDF, TextRank, BERTSum** are standard extractive methods, ordered by sophistication.
3. Modern abstractive systems use pre-trained Transformers (BART, PEGASUS) or LLMs (GPT-4, Claude). Decoder-only LLMs work because scale creates emergent ability and the prompt acts as the encoder.
4. **Hallucination** is the central risk of abstraction.
5. **ROUGE** measures *n*-gram overlap (surface-level). **BERTScore** captures semantic similarity. **METEOR** adds synonym awareness. **FactCC / QuestEval** check faithfulness.
6. No single metric is sufficient. Always report multiple complementary metrics.